

A PROJECT REPORT ON

**HANDWRITTEN DEVANAGARI
CHARACTER RECOGNITION USING
MACHINE LEARNING**

**SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE**

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

**VIRAAG BORUNDIYA
PARTH DAWANDE
DARPAN KOTAGIRE
SNEHA BASU**

**Exam No: B400050349
Exam No: B400050368
Exam No: B400050456
Exam No: B400050603**



**DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043**

**SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025**



CERTIFICATE

This is to certify that the project report entitled
Handwritten Devanagari Character Recognition Using Machine Learning

Submitted by

VIRAAG BORUNDIYA
PARTH DAWANDE
DARPAN KOTAGIRE
SNEHA BASU

Exam No: B400050349
Exam No: B400050368
Exam No: B400050456
Exam No: B400050603

are bonafide students of this institute and the work has been carried out by them under the supervision of **Asst. Prof. Samadhan Jadhav** and **Asst. Prof. Achala Deshpande** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering (Computer Engineering)**.

Asst. Prof. S.W. Jadhav
Guide
Dept. of Computer Engg.

Dr. G. V. Kale
Head
Dept. of Computer Engg.

Dr. S. T. Gandhe
Principal
Pune Institute of Computer Technology

Place: Pune
Date: 2/4/2025

ACKNOWLEDGEMENT

*It gives us great pleasure in presenting the project report on ‘**Handwritten Devanagari Character Recognition Using Machine Learning**’.*

*We would like to thank our project guide, **Asst. Prof. Samadhan Jadhav** and **Asst. Prof. Achala Despande**, for giving us all the help and guidance we needed. We are truly grateful to them for their kind support and valuable suggestions throughout the development of this project.*

*We are also grateful to **Dr. G. V. Kale**, Head of the Computer Engineering Department, PICT, for providing us with the necessary resources and a supportive academic environment that enabled us to carry out this project smoothly. We also express our sincere gratitude to **Dr. A. R. Deshpande**, BE Project Coordinator, for her timely guidance and continued support.*

VIRAAG BORUNDIYA
PARTH DAWANDE
DARPAN KOTAGIRE
SNEHA BASU
(B.E. Computer Engg.)

ABSTRACT

This report presents the implementation of a web-based system designed to analyze handwritten Devanagari script using advanced machine learning techniques. The system integrates three core functionalities—handwritten spell checking, inaccuracy detection in letter formation, and character recognition—into a cohesive pipeline. The Spell Checker module processes images of handwritten Hindi text to identify potentially incorrect words and suggest appropriate corrections through contextual analysis. Simultaneously, the Inaccuracy Detector evaluates the structural integrity of individual characters, providing confidence scores and highlighting poorly formed or ambiguous shapes. The Character Recognition component leverages deep learning models, particularly Convolutional Neural Networks (CNNs) and EfficientNetB0, to accurately recognize and classify handwritten Devanagari characters. The web application is developed using a React frontend for user-friendly interaction, a Node.js/Express.js backend for efficient processing, and MongoDB for data storage. This project highlights the potential of machine learning to enhance optical character recognition (OCR) systems, educational platforms, and the digital processing of Devanagari script.

Contents

1	Introduction	1
1.1	Overview	2
1.2	Motivation	2
1.3	Problem Definition and Objectives	3
1.4	Project Scope and Limitations	3
1.5	Methodology of Problem Solving	4
2	Literature Survey	5
2.1	Literature Survey	6
3	Software Requirements Specification	8
3.1	Assumptions and Dependencies	9
3.2	Functional Requirements	9
3.2.1	Handwritten Devanagari Character Recognition	9
3.2.2	OCR-Based Text Extraction	9
3.2.3	Inaccuracy Detection and Confidence Scoring	10
3.2.4	Spell Checker with Correction Suggestions	10
3.3	External Interface Requirements	10
3.3.1	User Interfaces	10
3.3.2	Hardware Interfaces	11
3.3.3	Software Interfaces	11
3.4	Non-Functional Requirements	12
3.4.1	Performance Requirements	12
3.4.2	Safety Requirements	12
3.4.3	Security Requirements	12
3.4.4	Software Quality Attributes	12
3.5	System Requirements	13
3.5.1	Database Requirements	13
3.5.2	Software Requirements	13
3.5.3	Hardware Requirements	13
3.6	Analysis Models: SDLC Model to Be Applied	14

3.6.1	SDLC Model	14
4	System Design	15
4.1	System Architecture	16
4.1.1	User Interface (Web Browser & React Application): . .	16
4.1.2	Application Server (Node.js Server):	16
4.1.3	Recognition Model:	16
4.1.4	Confidence Scorer:	16
4.1.5	Spell Checker:	17
4.1.6	Database (MongoDB):	17
4.2	Data Flow Diagrams	18
4.3	Activity Diagram	19
4.4	Use Case Diagram	20
4.5	Class Diagram	20
4.6	Sequence Diagram	21
5	Project Plan	23
5.1	Project Estimate	24
5.1.1	Estimates	24
5.1.2	Reconciled Estimates	24
5.2	Risk Management	25
5.2.1	Risk Identification	25
5.2.2	Risk Analysis	25
5.2.3	Overview of Risk Mitigation, Monitoring, Management	26
5.3	Project Schedule	27
5.3.1	Project Task Set	27
5.3.2	Timeline Chart	27
5.4	Team Organization	28
5.4.1	Team structure	28
5.4.2	Management reporting and communication	28
6	Project Implementation	29
6.1	Overview of Project Modules	30
6.2	Tools and Technologies Used	30
6.3	Model Selection	30
6.4	Algorithm Details	31
6.4.1	Algorithm 1 – Devanagari Handwritten Character Recognition	31
6.4.2	Algorithm 2 – OCR and Confidence Scoring using Easy-OCR	32
6.4.3	Algorithm 3 – Spell Checking and Suggestion	32

6.4.4	Algorithm 4 – Model Training	33
7	Software Testing	35
7.1	Type of Testing	36
7.1.1	Alpha Testing	36
7.1.2	Beta Testing	36
7.2	Integration Testing	36
7.2.1	Objectives	37
7.2.2	Integration Testing Approach	37
7.2.3	Sample Test Cases	37
7.2.4	Results	38
7.3	Test cases & Test Results	38
7.3.1	Test Cases	38
8	Results	40
8.1	Outcomes	41
8.2	Screenshots	42
8.3	Inaccuracy Detector	42
8.4	Devanagiri Character Recogniser	42
8.5	Hindi Spell Checker	42
9	Conclusions	47
9.1	Conclusions	48
9.2	Future Work	48
9.3	Applications	48
10	Appendix	50
10.1	Appendix B	52
10.1.1	Survey Paper Details	52
10.1.2	Implementation Paper Details	53
10.2	Appendix C	54
10.2.1	Plagiarism Report	54
11	References	55

List of Figures

3.1	SDLC Model Diagram	14
4.1	System Architecture Diagram	17
4.2	Data Flow Diagram	18
4.3	Activity Diagram	19
4.4	Use Case Diagram	20
4.5	UML Diagram	21
4.6	Sequence Diagram	22
5.1	Gantt Chart	27

List of Tables

7.1	Test Cases for Devanagari Handwritten OCR System	39
-----	--	----

CHAPTER 1

INTRODUCTION

1.1 Overview

This project presents an integrated system designed to process and refine handwritten and printed Devanagari script from images. It combines deep learning-based character recognition, optical character recognition (OCR), and natural language processing (NLP) to build a pipeline that can extract, evaluate, and correct Devanagari text. The system supports Hindi and Marathi languages and consists of three key modules: a handwritten character recognition model, a Devanagari inaccuracy detector that assigns confidence scores to extracted text, and a spell-checker that corrects linguistic errors. Together, these modules enhance the accuracy and reliability of Devanagari text processing, especially for handwritten input, which often suffers from legibility issues. The solution is deployed using a user-friendly web interface built with Streamlit, making it accessible for various practical applications such as educational content digitization, historical document archiving, and language learning support. This project addresses a critical need for accurate, automated processing of Indian scripts.

1.2 Motivation

In recent years, a noticeable decline has been observed in students' ability to fluently read and write in native Indian languages that use the Devanagari script, such as Hindi, Marathi, and Sanskrit. Despite receiving formal education in both English and their mother tongue, many students tend to favor English in written communication, leading to a gradual erosion of their script proficiency. This shift is largely due to the dominance of digital tools and typing in Roman script, which often replace traditional handwriting practices. As a result, students struggle with accurate spelling, grammar, and script formation in Devanagari.

This project is motivated by the desire to support language learning and script literacy, especially among younger students. By providing a tool that can recognize handwritten Devanagari text, detect inaccuracies, and suggest correct spellings, we aim to make learning these scripts interactive and supportive. Moreover, the system can serve as a valuable aid for teachers, helping them assess students' work more efficiently. It also supports broader goals such as the preservation of linguistic heritage, improved literacy in regional languages, and the integration of AI to enhance traditional education systems.

1.3 Problem Definition and Objectives

In the current educational landscape, students are increasingly struggling with the ability to fluently read and write in Devanagari script, which forms the foundation of several Indian languages like Hindi, Marathi, and Sanskrit. Although these languages are part of school curricula, the growing reliance on English and digital tools that favor Roman script has led to a steady decline in the use and understanding of Devanagari. As a result, students often face difficulties with spelling, handwriting clarity, and script formation. Traditional teaching methods lack the ability to offer immediate feedback or corrections, and existing OCR tools fall short when it comes to supporting handwritten Indian scripts. There is a clear need for a system that can accurately recognize Devanagari characters from images, detect errors, and assist in correcting them to support students' learning and promote native language literacy.

To address this need, the objective of this project is to create a comprehensive tool that combines Devanagari character recognition, inaccuracy detection, and spell checking. The system aims to develop a robust machine learning model capable of identifying handwritten characters, along with an OCR-based inaccuracy detector that provides character-level confidence scores. Furthermore, a spell-checking module is integrated to suggest correct spellings for any misidentified or misspelled words. The solution supports both Hindi and Marathi languages and is accessible through a user-friendly Streamlit web application. By combining deep learning, natural language processing, and intuitive design, the project not only helps in digitizing Devanagari content but also serves as an educational tool for students and teachers, promoting script literacy and reducing language learning barriers.

1.4 Project Scope and Limitations

This project aims to develop an end-to-end system for recognizing, evaluating, and correcting Devanagari script from images, with a focus on handwritten and printed Hindi and Marathi text. The system comprises three main modules: a character recognition model that identifies handwritten Devanagari letters, an inaccuracy detector that uses OCR to extract text and assign confidence scores to highlight potential recognition errors, and a spell-checker that analyzes the extracted text and provides spelling suggestions. The project supports language learning and educational use cases, document digitization, and preservation of regional scripts. It is designed to be user-friendly, accessible through a web interface built using Streamlit,

and suitable for both students and educators. The integration of EasyOCR, a custom recognition model, and the Gemini API enhances the system's versatility and performance.

However, the project does have certain limitations. The accuracy of recognition may vary depending on handwriting quality, image clarity, and the complexity of the script. Since the model has been trained on specific datasets, it may struggle with highly stylized or regional handwriting variations. The spell-checker relies on predefined language models and may not always suggest contextually accurate corrections. Additionally, the current system is limited to Hindi and Marathi, and expanding it to include other Devanagari-based languages like Sanskrit or Nepali would require further training and integration. Despite these limitations, the project lays a strong foundation for enhancing Devanagari script literacy and developing intelligent tools for regional language processing.

1.5 Methodology of Problem Solving

The project follows a modular and iterative approach to solve the problem of recognizing, evaluating, and correcting handwritten Devanagari script. The process begins with data preprocessing and model training for character recognition. A convolutional neural network (CNN) is trained on a dataset of handwritten Devanagari characters to enable accurate identification of individual script elements. This forms the base of the recognition system. Next, to handle real-world inputs such as scanned documents or photos, the project integrates optical character recognition (OCR) using EasyOCR, which extracts Devanagari text from images in Hindi and Marathi. A custom inaccuracy detector is developed using OCR output to assign confidence scores to each character, helping flag errors or ambiguities in recognition.

Following extraction and confidence analysis, the recognized text is passed through a spell-checking module, which uses natural language processing tools and the Gemini API to identify and correct spelling mistakes. This ensures that the final output is not only visually accurate but also linguistically meaningful. Finally, all components are brought together in a unified, web-based application using Streamlit, allowing users to upload images, view recognition results, and receive correction suggestions in real time. This stepwise, integrated methodology ensures accuracy, accessibility, and adaptability, making the system useful in educational and archival contexts.

CHAPTER 2

LITERATURE SURVEY

2.1 Literature Survey

Handwriting recognition in the Devanagari script poses unique challenges due to its complex structural components and variations in individual handwriting styles. Several machine learning approaches have been proposed and studied to detect and correct errors in handwritten Devanagari text.

Satish Kumar [?] introduced an early system for analyzing irregularities in Devanagari handwriting, emphasizing the importance of vertical segmentation (upper, middle, lower zones) to better manage script complexity. However, the approach struggles with incomplete characters and inconsistent symbol sizes, limiting its performance in real-world applications.

Mahesh Jangid and Sumit Srivastava [?] demonstrated the effectiveness of Convolutional Neural Networks (CNNs) for character recognition, achieving high accuracy under ideal conditions. Despite their success, CNNs showed vulnerability to noisy inputs and varying handwriting styles. The study suggests that incorporating data augmentation and noise reduction can improve robustness.

Amani Ali Ahmed Ali and Suresha Mallaiah [?] proposed a hybrid CNN-SVM model for Arabic script recognition. The CNN component extracts features, while the SVM performs classification. Although this hybrid model improves accuracy and generalization, it is resource-intensive and requires a large dataset, making it less ideal for low-resource environments. Its architecture holds potential for adaptation to Devanagari script.

Aditi Moudgil et al. [?] applied Capsule Networks (CapsNets) for Devanagari manuscript character recognition. CapsNets preserve spatial relationships in the data, helping them outperform traditional CNNs in distorted or cursive handwriting scenarios. Nevertheless, they demand substantial labeled data and can struggle to generalize across diverse handwriting styles.

Jumoke Falilat Ajao et al. [?] used CapsNets for offline Yoruba word recognition. Their work, though focused on a different language, demonstrated that CapsNets could effectively handle scripts with diacritics, drawing parallels to the challenges of Devanagari. Limited data availability remained a significant barrier to broader adoption.

Manoj Kumar Gupta et al. [?] focused on identifying commonly occurring character patterns, especially in single-letter Devanagari words. By reducing the recognition scope to frequent symbols in the middle zone, their method enhanced efficiency. However, it lacked scalability for more complex and longer text inputs.

Damodar Magdum et al. [?] developed a rule-based system to correct invalid Devanagari words for text-to-speech (TTS) applications. Though primarily

intended for speech synthesis, their grammar-based error correction technique could be adapted to OCR and handwriting recognition systems for better phonetic and structural accuracy.

Additional contributions include the work of Gupta and Yadav [?], who implemented SVM-based recognition for handwritten Devanagari characters, and Sarkar and Roy [?], who developed deep CNN architectures for Devanagari word-level recognition. Singh and Mehta [?] focused on morphological operations for segmentation, while Patel and Patel [?] reviewed various machine learning approaches for script recognition. Kumar and Joshi [?] conducted a comparative study of OCR techniques specific to Devanagari, and Bhattacharjee and Dey [?] explored hybrid models for detecting Devanagari text in natural scenes.

In summary, while CNNs and CapsNets have shown promising results in feature extraction and classification, hybrid approaches and rule-based error correction offer additional robustness. However, all models face limitations in dealing with diverse and noisy data. Future research should aim at building data-efficient, generalized models capable of handling cursive, distorted, or incomplete Devanagari characters.

CHAPTER 3

**SOFTWARE REQUIREMENTS
SPECIFICATION**

3.1 Assumptions and Dependencies

This project is built under the assumption that the input images provided will contain clear and legible Devanagari script, either handwritten or printed, in Hindi or Marathi. It is assumed that the images will be of moderate quality, with sufficient resolution to enable accurate OCR processing. The system relies on the presence of distinct character spacing and proper lighting in the image to achieve optimal recognition accuracy. Another assumption is that users have basic familiarity with uploading files and interacting with a web-based interface.

The project has several key dependencies. It depends on the EasyOCR library for text extraction and the Gemini API for advanced language processing and spell-checking. The spell-check functionality also relies on a predefined vocabulary and language model that may not account for highly regional or uncommon words. Additionally, the web interface is built using Streamlit, and the system requires standard Python libraries including TensorFlow, NumPy, PIL, and others for image handling and model execution. Lastly, a stable internet connection is required to access Gemini services during runtime.

3.2 Functional Requirements

3.2.1 Handwritten Devanagari Character Recognition

The system must accurately identify and classify handwritten characters from the Devanagari script, focusing on Hindi and Marathi. This is achieved using a convolutional neural network (CNN) trained on a labeled dataset of Devanagari characters. Upon receiving an image input, the model isolates and processes each character to determine the most probable matching class. This feature addresses the wide variation in handwriting styles and enables digitization of physical educational content, personal notes, and traditional texts written in Devanagari script. It supports preprocessing techniques such as grayscale conversion, resizing, and normalization to enhance recognition accuracy.

3.2.2 OCR-Based Text Extraction

This module uses the EasyOCR library to extract textual data from input images. It handles both printed and handwritten Devanagari content, supporting multi-line text, various fonts, and images with moderate distortion

or noise. It reads entire lines or paragraphs and outputs raw text, which can then be refined. This OCR module is the foundation of the system, converting visual data into machine-readable form and feeding it to subsequent modules for analysis and correction.

3.2.3 Inaccuracy Detection and Confidence Scoring

Once the text is extracted via OCR, this module evaluates the recognition confidence of each character or word. Using the confidence scores generated by the OCR engine, it flags uncertain recognitions that fall below a predefined threshold. This helps users identify potential errors early and intervene manually if needed. The feature is especially valuable in educational or archival scenarios where content accuracy is critical. Highlighting low-confidence segments adds a layer of transparency and allows users to verify or adjust the results accordingly.

3.2.4 Spell Checker with Correction Suggestions

This component analyzes the extracted text for spelling errors using natural language processing (NLP) techniques. It identifies incorrect words and suggests contextually appropriate corrections based on a language model and vocabulary dataset. Additionally, it leverages the Gemini API to provide intelligent correction suggestions that consider sentence context, not just word-level accuracy. This feature is crucial for reinforcing correct spelling among learners and ensuring that the output text is linguistically valid and usable for downstream tasks such as digital storage or publishing.

3.3 External Interface Requirements

3.3.1 User Interfaces

The system provides a clean and intuitive web interface developed using React.js. It is designed to cater to a wide range of users including students, educators, researchers, and general language learners. Users can upload images in formats such as .png, .jpg, or .jpeg through a simple drag-and-drop feature or a file picker interface. Once the image is uploaded, the interface displays the predicted characters in a readable text format. Additional visual components such as confidence scores, bounding boxes (optional), and highlighted uncertainties are included for better feedback. Progress indicators

such as loading spinners and bars keep users informed during model processing. The system also handles errors gracefully, showing helpful messages for unsupported formats or server issues. The interface is fully responsive and adapts seamlessly to desktops, tablets, and mobile screens, ensuring accessibility across platforms. Accessibility features such as semantic HTML, alt text, and form labels ensure a user-friendly experience for visually impaired users.

3.3.2 Hardware Interfaces

While the Devanagari Detection System primarily functions as a web-based application, it does rely on certain hardware configurations—particularly on the server side—to ensure efficient model inference and image processing. On the client side, no special hardware is required beyond a standard internet-enabled device (desktop, laptop, tablet, or smartphone) and a modern browser. On the server side, the application requires a machine capable of running the backend services and executing the recognition model. This can be a physical server or a virtual machine. For optimal performance, especially when processing high-resolution images or handling batch requests, GPU support (such as CUDA-enabled NVIDIA GPUs) is recommended. Minimum specifications include a quad-core CPU (e.g., Intel i5 or Ryzen 5), at least 8 GB RAM, and SSD storage if images are stored locally.

3.3.3 Software Interfaces

The Devanagari Letter Detection System integrates several software components to manage the end-to-end workflow of image processing and character recognition. The frontend, built using React.js, allows users to upload images and displays recognition results. It communicates with the backend via HTTP requests using libraries like Axios.

The backend is developed using Node.js and Express.js, which handle API routing, image upload (using Multer), and communication with the machine learning model. This model, built using frameworks like TensorFlow, Keras, or PyTorch, can be triggered either by executing a Python script or by calling a separate API hosted on Flask or FastAPI.

For optional data storage, MongoDB is used to log predictions and image metadata. The Mongoose library helps define and interact with the database schema. JSON is used as the standard format for data exchange, and temporary file system storage is employed to hold uploaded images during processing.

3.4 Non-Functional Requirements

3.4.1 Performance Requirements

The system should be capable of delivering fast and consistent performance to ensure a smooth user experience. Image upload, OCR processing, character confidence evaluation, and spell correction should all be completed within a few seconds. The application must support single-image processing without noticeable lag and should be responsive even on mid-range hardware. Latency should remain minimal when interacting with external APIs like Gemini. In educational settings, the tool must allow repeated use without requiring system restarts or manual resets.

3.4.2 Safety Requirements

While the system does not directly involve physical components or critical life-support systems, safety in the context of user interaction and data handling is important. The application should avoid crashes, freezing, or sudden shutdowns that may cause users to lose progress. Proper error handling must be implemented so that the system can gracefully respond to unsupported file types, failed API requests, or unexpected input without causing system instability. In educational environments, this helps ensure a dependable and frustration-free experience.

3.4.3 Security Requirements

The system must ensure that all uploaded images and extracted data are processed securely. No user data should be stored on the server or shared with third parties without explicit permission. API keys used for external services like Gemini must be stored securely and authenticated properly. The system should also validate inputs to prevent injection attacks or abuse. Since the application may be used by students and institutions, compliance with basic data privacy and security standards is essential.

3.4.4 Software Quality Attributes

The system should be designed with high maintainability, reliability, and usability in mind. Modularity is crucial—components like OCR, confidence scoring, and spell checking should be independently replaceable or upgradable. The system must be reliable, recovering gracefully from minor errors without data loss. It should be portable across systems and compatible with

common browsers and platforms. Finally, usability must remain a top priority, with a clear, intuitive interface suitable for both tech-savvy users and educators or students with minimal technical background.

3.5 System Requirements

3.5.1 Database Requirements

In this project, two types of data storage mechanisms are used. The OCR results, including extracted Devanagari text, confidence scores, and spell-check outputs, are handled in-memory within the Streamlit session. These are temporary and session-specific, ensuring quick access and processing without persistent storage unless explicitly exported. On the other hand, all user-related information such as login and signup details are stored in a MongoDB database. MongoDB provides a flexible schema design, which is ideal for managing user authentication data securely and efficiently. It allows easy integration with Python backends and offers scalability if the application expands to support a larger user base.

3.5.2 Software Requirements

The system is built using Python, leveraging its extensive libraries for deep learning, image processing, and web app development. Core components include TensorFlow/Keras for character recognition, EasyOCR for text extraction, and Streamlit for the front-end interface. The spell-check module is powered by the Gemini API. The platform is OS-independent and can be executed on Windows, Linux, or macOS environments that support Python 3.8 or higher.

3.5.3 Hardware Requirements

Minimum hardware requirements include a system with at least 8 GB RAM, an Intel i5 or equivalent processor, and 10 GB of free storage. For enhanced performance, especially during model training or when processing large images, GPU acceleration is recommended. For deployment on a cloud or local server, a machine with 2 vCPUs and 4–8 GB RAM is sufficient for handling concurrent users and real-time processing tasks.

3.6 Analysis Models: SDLC Model to Be Applied

The Software Development Life Cycle (SDLC) is a structured process that helps in planning, developing, and maintaining software effectively. For the handwriting error detection system for Devanagari script, we will use the Agile SDLC model. This approach is flexible and allows for quick adjustments based on user feedback. Below, we outline the chosen SDLC model, its phases, and how it will be applied in this project.

3.6.1 SDLC Model

An iterative SDLC model was used in this project as the model and its use cases expanded according to the requirement analysis and gatherings:

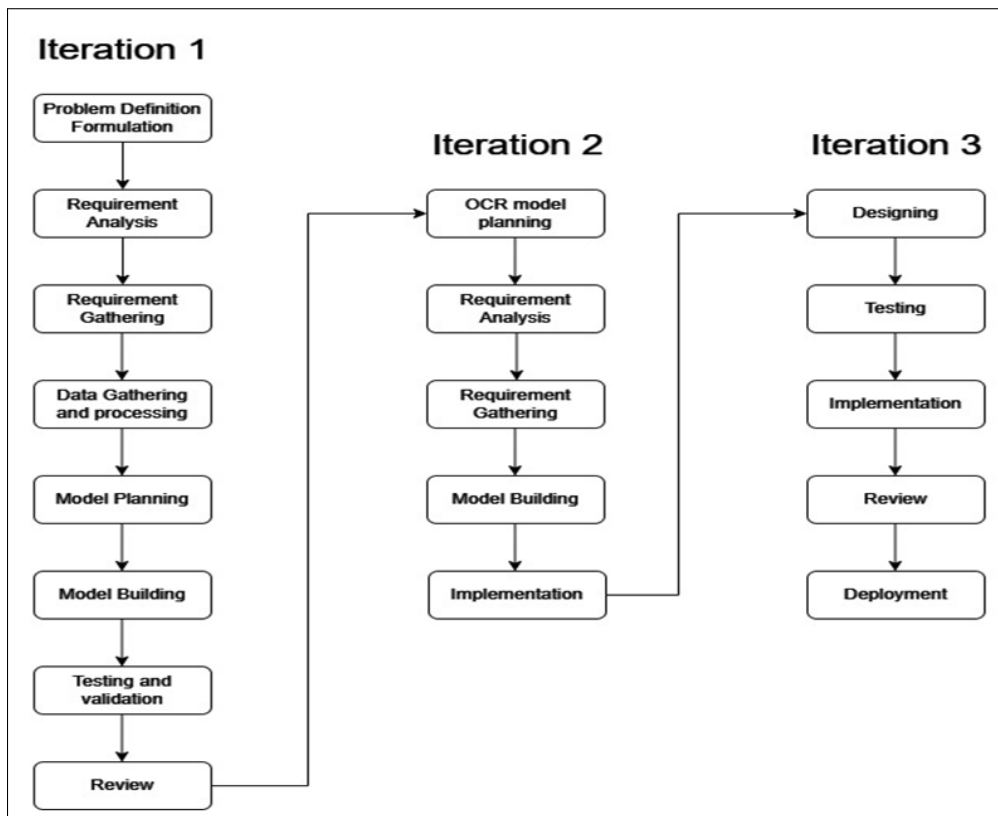


Figure 3.1: SDLC Model Diagram

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

The system is designed using a modular client-server architecture that separates the user interface from the processing logic and data storage. This architectural approach ensures scalability, maintainability, and a smooth user experience. The overall architecture consists of the following key components:

4.1.1 User Interface (Web Browser & React Application):

The user interacts with the system through a web browser that hosts a React-based front-end application. This interface allows users to upload images containing Devanagari script and then view the recognized and corrected text along with character-level confidence scores. The React application ensures responsiveness and a seamless experience, making it accessible even to non-technical users such as school students and educators.

4.1.2 Application Server (Node.js Server):

The Node.js server functions as the core orchestrator of the application. It handles incoming image uploads from the frontend and coordinates backend processes including text recognition, confidence scoring, and spell checking. It also manages routing, error handling, and efficient communication between all subsystems.

4.1.3 Recognition Model:

This module performs the core task of recognizing and classifying handwritten Devanagari characters from the uploaded image. It utilizes a deep learning-based model, trained on labeled datasets of Devanagari characters. The model analyzes the image input and returns the most likely character sequences based on learned features and visual patterns.

4.1.4 Confidence Scorer:

Once the text is extracted from the image, it is passed to the confidence scoring module. This component evaluates each recognized character or word

and assigns a confidence score indicating its recognition accuracy. Characters with low scores are flagged, guiding users to manually verify or correct potentially inaccurate results.

4.1.5 Spell Checker:

The spell-checking module further refines the extracted text by identifying spelling errors and suggesting corrections. It uses natural language processing (NLP) techniques along with the Gemini API to generate context-aware suggestions. This ensures grammatical accuracy and improves the overall quality of the final output.

4.1.6 Database (MongoDB):

MongoDB is used as the primary data store for the system. It stores user inputs, image metadata, recognition results, and correction history. This enables persistent data storage and supports future analysis and feedback-driven improvements. The database structure also facilitates tracking and refining spelling suggestions over time based on user interaction patterns.

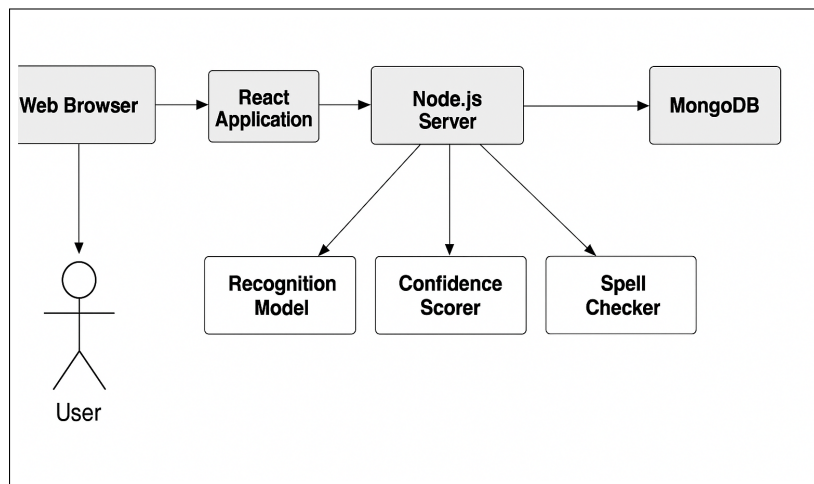


Figure 4.1: System Architecture Diagram

4.2 Data Flow Diagrams

The data flow diagram illustrates the movement of data throughout the system. Users upload an image, which is processed by the Character Recognition module using a pre-trained Character Dataset. Extracted text is then analyzed through Spell Checking and Inaccuracy Detection modules. Confidence scores help flag errors, and corrections are made using a Language Model, resulting in the final corrected Devanagari text output.

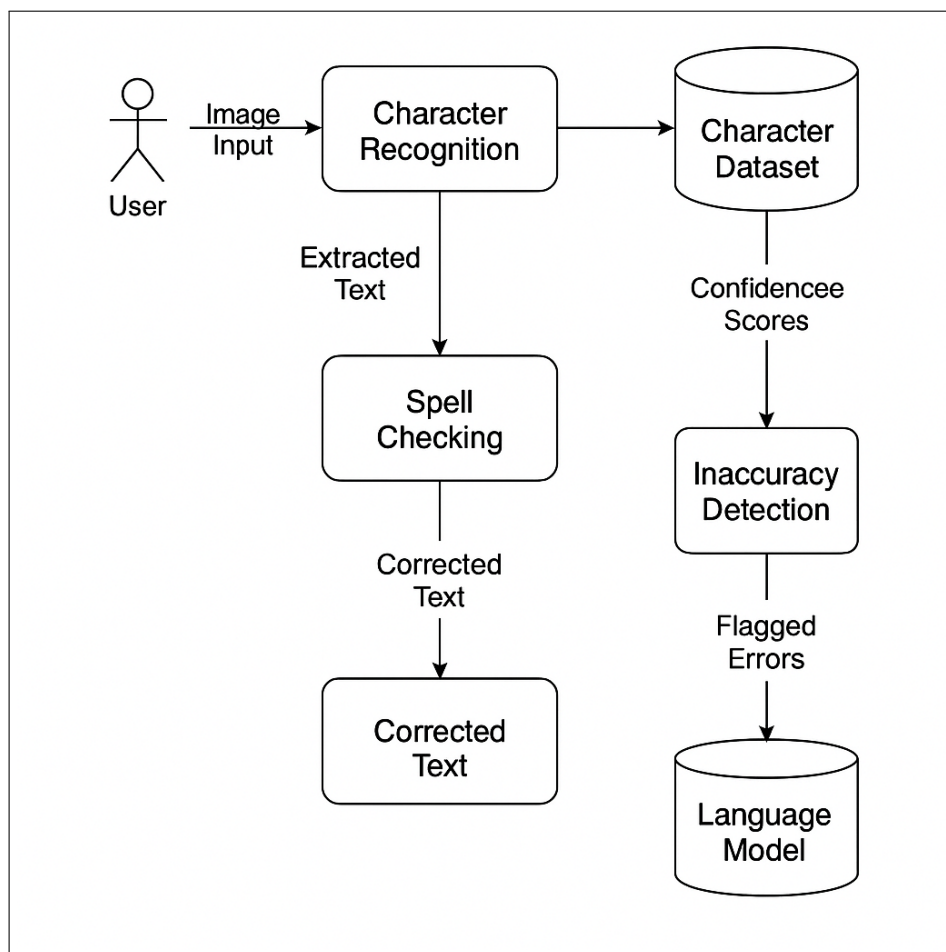


Figure 4.2: Data Flow Diagram

4.3 Activity Diagram

The activity diagram illustrates the sequential flow of operations in the system. It starts with the user uploading an image, followed by preprocessing and character segmentation. Characters are recognized using a CNN model, and confidence is evaluated. If low confidence is detected, spelling is checked and corrections are suggested using the Gemini API. Finally, the corrected result is presented to the user for review.

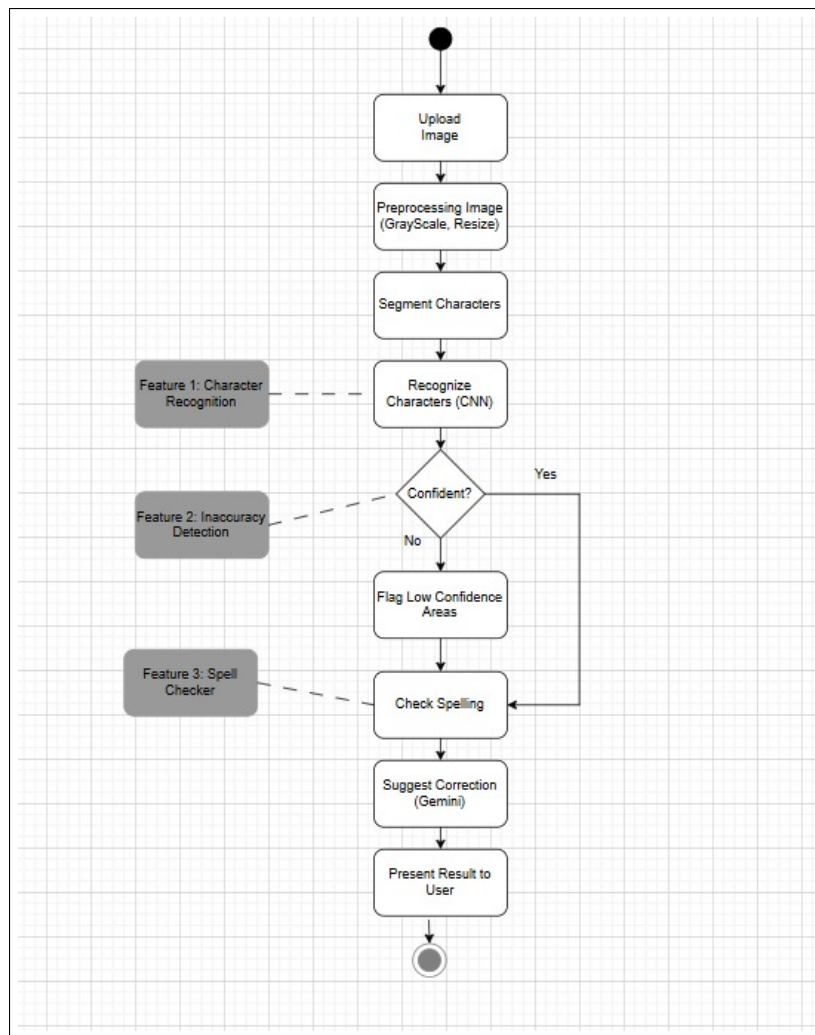


Figure 4.3: Activity Diagram

4.4 Use Case Diagram

The use case diagram represents the interaction between the user and the Devanagari OCR system. The user uploads images, initiates character recognition, checks the detected accuracy, verifies spellings, and finally reviews and exports the corrected text. Each action triggers corresponding system operations. This diagram clearly outlines the main functionalities the system provides and how the user interacts with each feature.

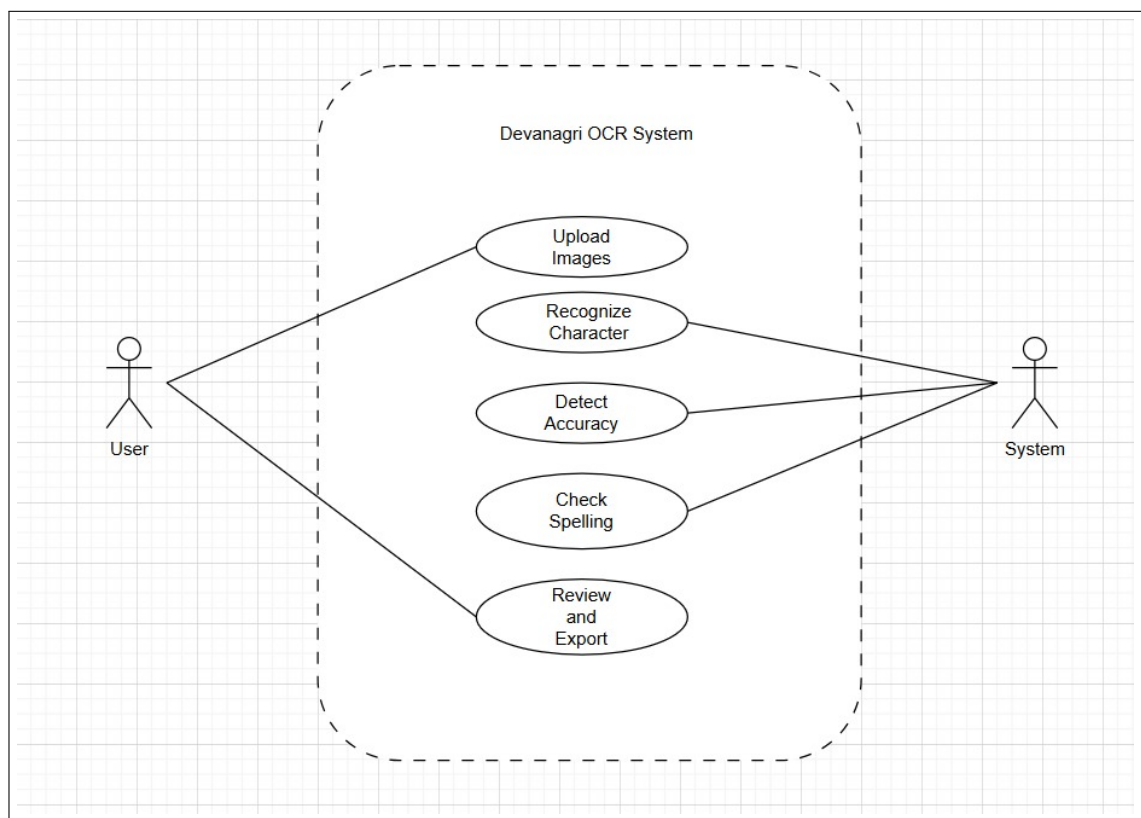


Figure 4.4: Use Case Diagram

4.5 Class Diagram

The class diagram represents the structure of the Devanagari OCR system. It defines classes such as User, ImageProcessor, OCRModule, SpellChecker, and Image, each with their respective attributes and methods. Relationships

between classes show how images are uploaded, processed, and reviewed. The OCRModule interacts with both the ImageProcessor and SpellChecker, highlighting the modular and interconnected design of the system.

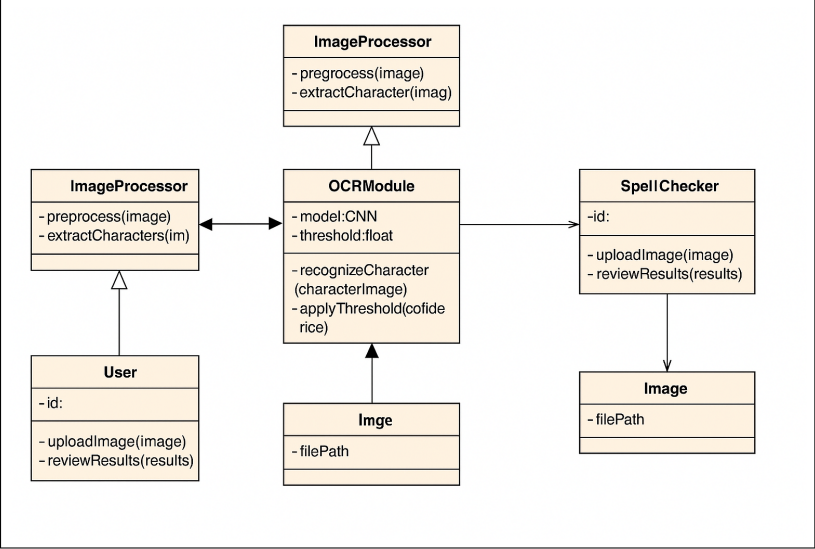


Figure 4.5: UML Diagram

4.6 Sequence Diagram

The sequence diagram illustrates the interaction between the user, web interface, OCR engine, and correction service. It begins with the user uploading an image through the interface, which is processed by the OCR engine to perform character recognition. Detected inaccuracies are passed to the correction service for spelling suggestions. The corrected results are then returned to the user through the web interface for review and display.

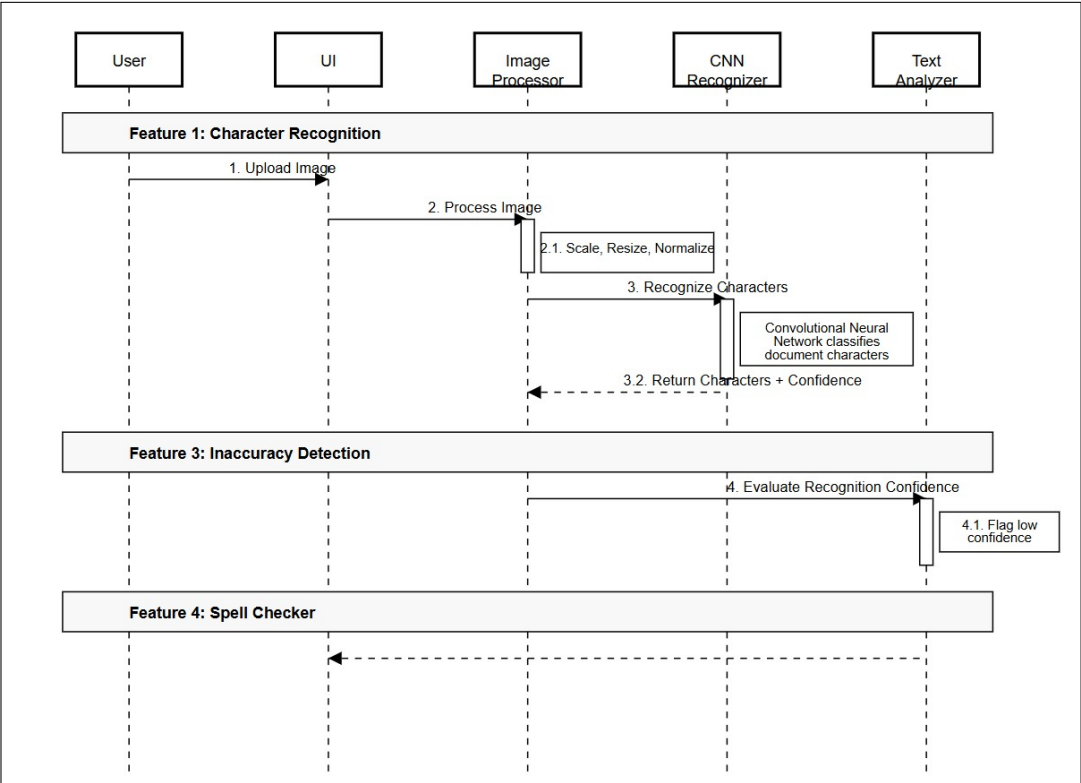


Figure 4.6: Sequence Diagram

CHAPTER 5

PROJECT PLAN

This section outlines the project plan for the machine learning-based handwriting error detection system for the Devanagari script. It includes estimates, risk management strategies, the project schedule, and team organization.

5.1 Project Estimate

5.1.1 Estimates

The timeline and resource estimates for the project have been carefully planned based on the scope of each individual module, the integration efforts involved, and the testing required to ensure a smooth and functional end-to-end system.

The first phase involves the development of the handwritten Devanagari character recognition model using a deep learning approach. This includes dataset collection and cleaning, preprocessing, model design, training, validation, and fine-tuning. This phase is estimated to take around 2–3 weeks depending on dataset size and model complexity.

Next, integrating the OCR functionality using EasyOCR and implementing the inaccuracy detection module with confidence scoring will take an estimated 1.5 to 2 weeks. This stage includes the extraction of text from images and the development of logic to highlight uncertain recognitions.

The third stage focuses on building the spell checker using natural language processing techniques and integrating it with the Gemini API. Designing a correction mechanism that can identify errors and suggest valid alternatives contextually will require approximately 1–1.5 weeks.

Following the backend functionality, developing the web-based frontend using Streamlit, including all user interactions, real-time processing feedback, and result display, is expected to take another 1–1.5 weeks.

Finally, 1–2 weeks are reserved for system integration, testing, debugging, UI refinement, performance optimization, and deployment preparation.

Overall, the entire project is estimated to take approximately 7 to 9 weeks of dedicated development time, assuming regular progress and minimal external delays. This estimate allows room for buffer time to handle unexpected issues, feature revisions, or additional polishing based on early user feedback.

5.1.2 Reconciled Estimates

After reviewing the individual module-wise estimates and accounting for overlaps, integration phases, and possible delays, the reconciled total estimate

for the project is approximately 8 to 9 weeks. While initial estimates for the handwritten character recognition, OCR integration, and spell-checker development suggested a cumulative timeline of around 6–7 weeks, additional time has been included to account for testing, system-level integration, user interface adjustments, and possible re-training or refinement of models based on real-world testing.

This reconciliation considers the fact that some tasks may run in parallel—for instance, frontend design can begin while the model is being trained. However, other tasks like integration and debugging are sequential and require that core modules be completed first. Furthermore, buffer time has been added to accommodate potential setbacks such as unexpected model behavior, UI optimization needs, or feedback-based iteration.

Hence, to ensure the system is functionally robust, accurate, and user-friendly by the time of completion, a total duration of 8 to 9 weeks is considered realistic and achievable for the full development cycle of the project.

5.2 Risk Management

5.2.1 Risk Identification

During the planning and development of this project, several potential risks have been identified that could affect its timeline, functionality, or performance. One of the key risks is the variability in handwriting styles, which can significantly impact the accuracy of the character recognition model. Poor-quality input images, including those with low resolution, noise, or lighting issues, may lead to inaccurate OCR output. Dependency on third-party APIs like Gemini also introduces risks, such as service outages, rate limitations, or changes in access policies. Additionally, integration of various modules—OCR, confidence scoring, spell-checking, and UI—may present technical challenges, particularly during testing and debugging. There is also the risk of limited dataset diversity, which could hinder the model’s generalizability to different user handwriting styles. Finally, a lack of user familiarity with Devanagari may affect the validation of recognition outputs during initial testing phases.

5.2.2 Risk Analysis

To mitigate these risks, several strategies have been incorporated into the project plan. For handling handwriting variation and dataset limitations, the model will be trained on a diverse and representative dataset, and data

augmentation techniques may be applied to improve generalization. Preprocessing steps such as resizing, grayscale conversion, and noise reduction will help manage variability in image quality. To reduce reliance on third-party APIs, the system will include fallback mechanisms or cached responses to handle temporary unavailability. Integration challenges will be addressed through modular development and regular unit testing to ensure that each component functions correctly before full system integration. Additionally, early-stage user testing will be conducted to gather feedback and identify edge cases, ensuring the system performs reliably under various real-world conditions. By anticipating these risks and planning accordingly, the project aims to minimize disruptions and ensure smooth, timely delivery of a functional and accurate Devanagari recognition and correction system.

5.2.3 Overview of Risk Mitigation, Monitoring, Management

Effective risk mitigation, monitoring, and management are essential to the successful delivery of this project. From the initial stages of development, a proactive approach has been taken to identify potential risks related to accuracy, integration complexity, third-party dependencies, and user-related variability. To mitigate these risks, the project follows a modular design approach, where each component—character recognition, OCR, inaccuracy detection, and spell-checking—is developed and tested independently before full system integration. This helps isolate issues early and reduces debugging complexity later.

Continuous monitoring is planned throughout the development cycle using regular testing checkpoints and performance evaluations. Each module's output will be validated with test cases and edge scenarios to ensure the system behaves as expected. Automated logging and exception handling will be implemented to capture runtime errors or API failures, allowing developers to respond promptly.

Risk management also includes planning for fallback strategies, such as handling API unavailability or low-confidence recognition results gracefully. Scheduled buffer time in the project timeline serves as a safety net to manage delays without compromising overall quality. Regular project reviews and progress tracking will ensure that risks are being managed proactively, and any emerging issues can be addressed promptly. This structured approach ensures a resilient development process and enhances the reliability and maintainability of the final system.

5.3 Project Schedule

5.3.1 Project Task Set

- Task 1: Project Planning and Scope Definition
- Task 2: Tool and Technology Selection
- Task 3: Data Collection
- Task 4: Data Annotation
- Task 5: Data Preprocessing
- Task 6: Model Selection
- Task 7: Initial Model Development
- Task 8: Model Training and Hyperparameter Tuning
- Task 9: Testing and Validation
- Task 10: UI Development
- Task 11: Integration and Deployment
- Task 12: Final Documentation and Reporting
- Task 13: Project Presentation

5.3.2 Timeline Chart

Gantt Chart

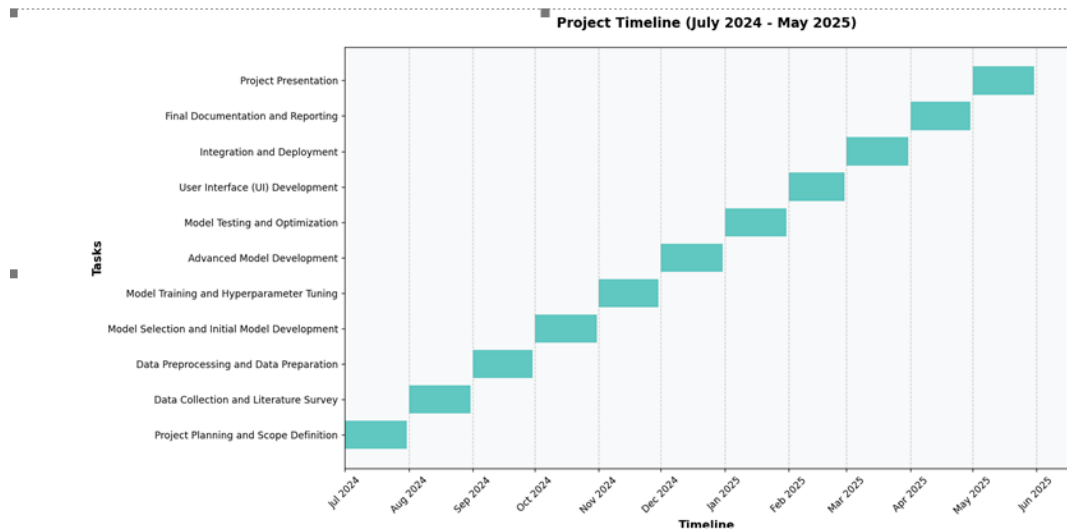


Figure 5.1: Gantt Chart

5.4 Team Organization

5.4.1 Team structure

- **Viraag Borundiya:** ML Developer, Backend Developer, Integration Lead.
- **Parth Dawande:** UI Designer, Frontend Developer.
- **Darpan Kotagire:** UI Designer, Frontend Developer.
- **Sneha Basu:** Concept Developer, ML Developer, Backend Developer.
- **Asst. Prof. S. Jadhav:** Internal mentor and guide.

5.4.2 Management reporting and communication

Regular Meetings: Weekly team meetings to discuss progress and address any challenges.

Project Management Tool: Python, Streamlit, EasyOCR, Gemini API, TensorFlow, Keras, NumPy, OpenCV, PIL, Pandas, Matplotlib, Jupyter Notebook, CNN, OCR, NLP, Spellchecker, Devanagari, Hindi, Marathi, Image Processing.

Documentation: Maintain shared documentation for all project aspects, ensuring all team members can access and contribute.

CHAPTER 6

PROJECT IMPLEMENTATION

6.1 Overview of Project Modules

The project consists of three main modules. The first module is responsible for recognizing handwritten Devanagari characters using a deep learning model trained on a labeled dataset. The second module extracts Devanagari script from uploaded images using an OCR system and provides confidence scores for each recognized segment to highlight possible inaccuracies in character formation. The third module functions as a spell checker that takes the recognized text and suggests corrections for misspelled or wrongly formed words. These modules are integrated into a single web application with a clean and intuitive interface, allowing users to upload Devanagari script images and receive immediate feedback on accuracy and spelling.

6.2 Tools and Technologies Used

- Python for backend development and model implementation
- TensorFlow and Keras for building and training the CNN recognition model
- EasyOCR for extracting Devanagari script from images
- Streamlit for developing the web-based user interface
- OpenCV and PIL for image preprocessing and manipulation
- Spacy and pypellchecker libraries for text processing and spell correction
- NumPy and Pandas for data handling and analysis

6.3 Model Selection

The model selection process for this project was guided by the need to accurately recognize handwritten Devanagari characters, which often display significant variation in style, size, and spacing. A Convolutional Neural Network (CNN) was chosen as the core architecture for character recognition due to its strong performance in image classification tasks. CNNs are well-suited for extracting spatial features from images and handling non-linear distortions common in handwritten inputs. The model was trained on a labeled dataset of Devanagari characters, and hyperparameters such as kernel size, number

of filters, and activation functions were carefully tuned for optimal accuracy and generalization. Additionally, data augmentation techniques were applied to improve the model's robustness to handwriting diversity. For the OCR component, EasyOCR was selected due to its built-in support for Hindi and Marathi scripts. This hybrid approach—using both a custom CNN for isolated character recognition and EasyOCR for word-level extraction—ensured greater flexibility and accuracy across use cases.

6.4 Algorithm Details

6.4.1 Algorithm 1 – Devanagari Handwritten Character Recognition

This module uses a Convolutional Neural Network (CNN) to classify handwritten characters. Input images are resized and normalized before being passed through convolutional and pooling layers. The feature maps are flattened and passed through dense layers, followed by a softmax output layer to predict the corresponding Devanagari character. The model is trained on a dataset of labeled characters to ensure accurate recognition across various handwriting styles.

```
# Preprocessing: Resizing and Normalization
X_train = X_train.reshape(-1, 32, 32, 1) / 255.0
X_test = X_test.reshape(-1, 32, 32, 1) / 255.0

# CNN Model for Devanagari Handwritten Character Recognition
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(32, 32, 1)),
    MaxPooling2D(2,2),
    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D(2,2),
    Flatten(),
    Dense(128, activation='relu'),
    Dense(46, activation='softmax') # Assuming 46 classes for Devanagari charac
])

# Model compilation
```



```
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

6.4.2 Algorithm 2 – OCR and Confidence Scoring using EasyOCR

This algorithm takes an input image and uses OCR to detect and extract text segments. For each recognized word or character, a confidence score is generated, representing the reliability of the recognition. If the score falls below a predefined threshold, the system flags the character as potentially inaccurate. This allows users or educators to review and validate the result, especially useful in cases of unclear handwriting or poor image quality.

```
import easyocr

# Initialize EasyOCR Reader
reader = easyocr.Reader(['hi', 'mr']) # Hindi and Marathi languages

# OCR + Confidence Scoring
def extract_text_with_confidence(image_path):
    results = reader.readtext(image_path)
    extracted_text = []

    for bbox, text, confidence in results:
        extracted_text.append({
            'text': text,
            'confidence': confidence
        })

    return extracted_text

# Example usage
# output = extract_text_with_confidence('path/to/image.jpg')
```

6.4.3 Algorithm 3 – Spell Checking and Suggestion

The spell checker processes the OCR-extracted text and splits it into individual words. Each word is checked against a built-in vocabulary or language model using token matching and edit-distance algorithms. If a word is not found or appears incorrect, the system suggests the closest valid alternatives. This module helps correct common spelling mistakes and refines the overall accuracy of the extracted text, especially in educational settings.

```
from spellchecker import SpellChecker

# Initialize the SpellChecker
spell = SpellChecker(language='hi') # or 'mr' if using Marathi-specific dictionary

def spell_check_text(text):
    corrected_text = []
    words = text.split()

    for word in words:
        if word not in spell:
            corrected = spell.correction(word)
            corrected_text.append(corrected)
        else:
            corrected_text.append(word)

    return ' '.join(corrected_text)

# Example usage
# output = spell_check_text('some incorrect text')
```

6.4.4 Algorithm 4 – Model Training

After building the CNN architecture for handwritten Devanagari character recognition, the next step is model training. The model is trained using labeled datasets containing images of Devanagari characters. During training, the model learns to minimize the loss function, adjusting its internal weights to improve classification accuracy. A validation set is used to monitor the model's performance and prevent overfitting. Early stopping and model checkpointing techniques can also be applied to save the best performing model.

```
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

# Define callbacks
early_stop = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
checkpoint = ModelCheckpoint('best_model.h5', save_best_only=True)

# Model Training
history = model.fit(
    X_train, y_train,
```

```
epochs=30,  
batch_size=64,  
validation_data=(X_test, y_test),  
callbacks=[early_stop, checkpoint])
```

The model is trained over multiple epochs with a batch size optimized for GPU/CPU performance. Validation accuracy and loss are monitored to determine when the model has sufficiently learned the character representations without overfitting.

CHAPTER 7

SOFTWARE TESTING

7.1 Type of Testing

7.1.1 Alpha Testing

Alpha testing was carried out internally by the development team to evaluate the core functionalities of the system in a controlled environment. The objective was to detect and fix bugs, performance bottlenecks, and functional inconsistencies before releasing the system to external users. During this phase, various modules—such as the Devanagari character recognition model, the OCR-based text extractor, the confidence scoring system, and the spell checker—were tested using a predefined set of handwritten and printed image inputs. Unit testing was conducted for each component individually, followed by integration testing to ensure smooth communication between modules. Test scenarios included checking how well the model recognized characters under varying handwriting styles and image quality, verifying the confidence thresholds, and ensuring that the spell checker returned contextually correct suggestions.

7.1.2 Beta Testing

Beta testing involved releasing the system to a select group of external users, including students, teachers, and native Devanagari script writers, to evaluate the tool in real-world conditions. The main focus was on usability, interface responsiveness, and practical accuracy of the system when dealing with everyday writing variations. Users interacted with the web application by uploading images and reviewing the OCR results, inaccuracy highlights, and suggested corrections. Feedback from this phase helped the development team identify practical challenges such as dealing with regional handwriting styles, false positives in spelling suggestions, and layout improvements for better user experience. Insights from beta testing were then used to make final adjustments to improve accuracy, robustness, and user satisfaction.

7.2 Integration Testing

Integration testing was carried out to verify the correct interaction and data flow between different modules of the Devanagari Handwritten Character Recognition system. It ensures that individual modules such as the user interface, application server, recognition model, confidence scorer, spell checker, and database work together as a unified system.

7.2.1 Objectives

- Verify that the image uploaded through the React frontend is correctly received and processed by the Node.js server.
- Ensure the Node.js server correctly invokes the recognition model, confidence scorer, and spell checker modules sequentially.
- Validate that the recognition model accurately processes input images and returns extracted character sequences.
- Confirm that confidence scores are properly generated and flagged for low-confidence recognitions.
- Ensure corrected text suggestions from the spell checker are properly integrated and returned to the frontend.
- Validate that recognized text, confidence scores, and correction history are stored appropriately in the MongoDB database.

7.2.2 Integration Testing Approach

- **Top-Down Testing:** Testing began at the React frontend and proceeded through the server, model inference modules, and database interactions.
- **Test Automation:** Integration tests were partially automated using unit tests for backend API routes and module responses.
- **Mocking External Dependencies:** External services such as the Gemini API for spell checking were mocked to isolate internal system behavior.

7.2.3 Sample Test Cases

- **Test Case 1:** Upload a sample handwritten Devanagari character image. Verify that the image is received by the server, passed to the recognition model, and the extracted text is displayed on the frontend.
- **Test Case 2:** Input an image with distorted handwriting. Check that the system flags low-confidence characters and displays appropriate warning indicators.
- **Test Case 3:** Simulate spelling errors in the recognized text. Verify that the spell checker suggests accurate corrections.

- **Test Case 4:** Disconnect the database. Attempt to upload an image and verify that appropriate error messages are shown without system crash.
- **Test Case 5:** Monitor MongoDB after multiple uploads to ensure proper storage of recognition history and user corrections.

7.2.4 Results

The integration testing confirmed that all modules communicated correctly, the processing pipeline was stable under normal operating conditions, and the system gracefully handled errors such as low-confidence recognitions and database failures. Minor issues related to API timeout were identified and fixed through asynchronous handling improvements.

7.3 Test cases & Test Results

7.3.1 Test Cases

Test Case ID	Input	Expected Output	Description	Result
TC01	Clear image with neatly written characters	Correct character recognition with high confidence scores	Test basic functionality of character recognition model with ideal input	Pass/Fail
TC02	Blurred or low-quality image	Lower confidence scores, some errors flagged	Verify system behavior on poor quality inputs and flagging mechanism	Pass/Fail
TC03	Image with cur-sive or overlap-ping characters	Partial recog-nition with medium con-fidence scores, correction suggestions trig-gered	Test model robustness to complex handwrit-ing styles	Pass/Fail
TC04	Correctly spelled words from OCR out-put	No corrections suggested	Validate that spell checker does not in-correctly modify valid words	Pass/Fail
TC05	OCR output with intentional spelling mis-takes	Correct sugges-tions from spell checker	Test effectiveness of spell checking and suggestions module	Pass/Fail
TC06	Upload empty or blank image	Appropriate er-ror message or no text recog-nized	Test system behavior on invalid/no input	Pass/Fail
TC07	Upload docu-ment with both Devanagari and English text	Recognize only Devanagari, ignore foreign scripts	Test language-specific recognition capability	Pass/Fail
TC08	Upload same image multiple times	Consistent recognition results across attempts	Check model stability and consistency	Pass/Fail

Table 7.1: Test Cases for Devanagari Handwritten OCR System

CHAPTER 8

RESULTS

8.1 Outcomes

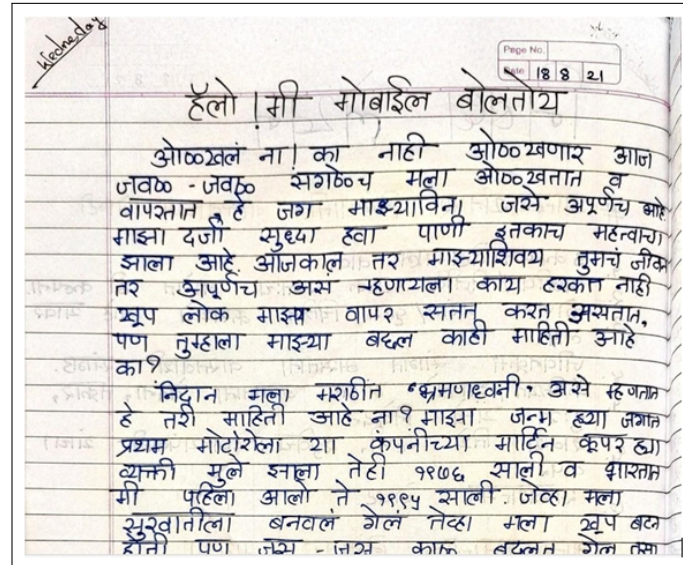
The Devanagari Letter Detection System achieved significant outcomes aligned with the problem statement. The following points summarize the major results observed:

- **Accurate Recognition of Devanagari Characters:** The system successfully recognizes handwritten Devanagari characters with high accuracy using a CNN model, minimizing manual effort in interpreting handwritten scripts.
- **Automated Confidence Scoring:** Each recognized character or word is assigned a confidence score, enabling users to easily identify sections that might require manual verification, thus enhancing reliability.
- **Enhanced Text Correction with Spell Checking:** The spell-checking module, based on NLP techniques, suggests corrections for misspelled or incorrectly recognized words, ensuring that the final output is both accurate and contextually meaningful.
- **Improved Learning for Students and Educators:** By flagging inaccuracies and suggesting corrections, the system helps students understand their mistakes and improve handwriting. Educators benefit from quicker, targeted feedback.
- **Robust System Design and Scalability:** The modular client-server architecture ensures scalability and maintainability, supporting future expansion to other Indian scripts and accommodating more users.
- **User-Friendly Interface for Non-Technical Users:** The React-based web application provides a clean, intuitive experience for uploading images, viewing results, and correcting errors without technical barriers.
- **Efficient Data Storage and Analysis:** Using MongoDB for storing inputs, results, and correction histories enables future training improvements and provides a valuable dataset for educational research on handwriting skills.

8.2 Screenshots

8.3 Inaccuracy Detector

Input:



Output:

8.4 Devanagari Character Recogniser

Input:

Output:

8.5 Hindi Spell Checker

Input:

Handwritten Devanagari Character Recognition Using Machine Learning

Extracted Text:

"० १810 १४ 8 २| साबाइल 72 आळखलं क नाहा ऊ१०ठ जवळ सगठ०च मता आखनान वापशत न जग माश्याविन ७२
अप्रच माझा सुथ्या हवा उतक= महतवाच झाला आजकाल कर माझ्याशवय तुमच जत अआर्थिच िस न्हणायन कर
हरकात नह तपैप लाक माझमा वापर सतन करत अथता उुन्हाला माझय बदल काही साहितती आह ट6 काड + २शठान
६भमणवनी अख ह निष्ठान माहिल्ली जाह ना माझया जनम हा खग7 ग९ प्रथाम साटाशल 2|| कंपनीच्या मार्टिन कूपर इनाना
तेही १९७५ साल याशता मीव्ती आला १९९५ साली जल रल पला बनटल नेव्हा मल २वप बर गन्वा जर जर बयलन गेल बर्लत
आण सक्रीनदृष्टया झाला आर चमवत माझ फकत करू शकन ठवढ च माझ नगपूदिन वापर नाह ७२ नळ्हत ७वट2
+ ०झ जादुई पेटित शुप काही त्वश्लेल आह इंटरनेट रेडिआ ठणी बासम्था 0 दऊ १०, जीपीनस किमेश रागारीम्मा
, ०ष्टरूमलयाह सवान आवडती ठ वड ल त्ोष्ट साड्यन आह न्हणज .सल्फ लोक सृणत्तात कि मी मुलान बिघडको =
शरन नायाल लावतय , उपथाग केडकोतक फर. अर्न अनक आशत मझ्ावर होतन माझा ३फ० उमहाला शुप < 3श ल
=री सझ ख अत वापर करताथ द९टू श80२ डल णर 35 वर आ. पाणी दजो पूणली कम० जेला २४ संकगणि ह= काह आण
कम उम्हा

Inaccuracies Detected:

Inaccuracy Report:

Character: 8, Position: 11, Predicted Label: 25, Confidence: 0.46
Character: , Position: 15, Predicted Label: 31, Confidence: 0.62
Character: स, Position: 16, Predicted Label: 15, Confidence: 0.67
Character: ा, Position: 17, Predicted Label: 31, Confidence: 0.76
Character: ब, Position: 18, Predicted Label: 39, Confidence: 0.62

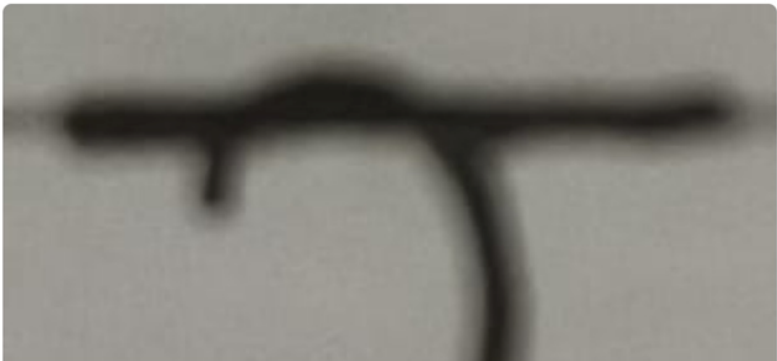
Devanagari Character Recognizer

Upload character image

 Drag and drop file here
Limit 200MB per file • PNG, JPG, JPEG

Browse files

 character_27_ra_correct.jpg 5.8KB ×



Recognize Character

Results

Predicted label: **character_27_ra**

Actual Label: character_27_ra

☒ Correctly written

Output:

Handwritten Devanagari Character Recognition Using Machine Learning

The use_column_width parameter has been deprecated and will be removed in a future release. Please utilize the use_container_width parameter instead.

Extracted Text:

डोरेमोन
जिन्दगी सवार हु, एक नही बहार हु
घरेरे मेनीम थे
थध नीध मेपर्म ग
दुनिया ही बदल ना मे तो प्यारा सा चमत्कार हु
नीरे म मेरे हैं नी थे पप गानीरंग ग

Found 63 potential spelling mistakes:

	Word	Line	Position	Suggested Correction
0	डोरेमोन	1	1	डोन
1	हु,	2	3	हुए
2	बहार	2	6	बार
3	हु	2	7	है
4	घरेरे	3	1	घरेलू
5	मेनीम	3	2	मेरी
6	थे	3	3	के

CHAPTER 9

CONCLUSIONS

9.1 Conclusions

This project successfully demonstrates a system that integrates Devanagari character recognition, inaccuracy detection, and intelligent spell correction using deep learning, OCR, and language models. The model is able to extract handwritten or printed Devanagari text from images and evaluate the accuracy of character formation through confidence scoring. Furthermore, the inclusion of a spell-checking module ensures that even incorrectly written words are flagged and corrected. Through internal and user testing, the system has shown promising results in terms of character accuracy and usability. The application can be instrumental in educational contexts, helping learners of Hindi, Marathi, and Sanskrit improve their writing skills. Overall, the project bridges a critical gap in digital tools available for native Indian language scripts.

9.2 Future Work

Future enhancements can involve training the recognition model on larger and more diverse datasets, including regional handwriting variations and cursive Devanagari styles. Incorporating support for more Indian languages that use the Devanagari script, like Konkani, Maithili, and Nepali, would also increase the utility of the tool. Another potential improvement includes adding real-time handwriting analysis for digital tablets or stylus-based inputs. Integrating speech-to-text and audio feedback could also make it more accessible to younger learners and visually impaired users. In addition, implementing user feedback loops to improve recognition accuracy over time, and incorporating grammar correction in addition to spelling, would strengthen the tool's educational value. Finally, deploying the system as a lightweight mobile application with offline capabilities would extend its reach to rural and low-resource settings where such digital educational support is most needed.

9.3 Applications

This system has several valuable applications. In education, it can assist students in improving their Devanagari handwriting by providing feedback and correction suggestions. Teachers can use it for automated grading or detecting common spelling errors in assignments written in Hindi or Marathi. It is also useful for digitizing historical or handwritten documents, making archival information more accessible. Additionally, language learn-

ers—especially children—can benefit from this as a guided learning tool to develop stronger Devanagari literacy skills. The system could be integrated into school curricula or interactive whiteboards for in-class learning. With further refinement, it could also become part of educational mobile apps, e-learning platforms, and assistive technologies that support inclusive, multilingual education.

CHAPTER 10

APPENDIX

Appendix A

Satisfiability Analysis

The core problem addressed in this project—recognizing and correcting handwritten Devanagari text—is a composition of classification, pattern recognition, and language correction tasks. These can be formalized as decision problems: for instance, “Is this sequence of strokes a valid Devanagari character?” or “Is this word correctly spelled in the given language context?” These problems are satisfiable if there exists at least one valid output (i.e., a correct label, spelling, or character match) that satisfies the constraints defined by the model’s learned parameters or dictionary rules.

In this context, the satisfiability conditions are not Boolean in the traditional SAT sense but rather defined over real-valued functions (e.g., activation functions in neural networks). Since models like CNNs use continuous activation and optimization over differentiable loss functions, solutions exist where the loss is minimized and accuracy is maximized, implying the problem is satisfiable under bounded resources and data.

Complexity Class Analysis

The recognition and spell correction tasks tackled in this project are computationally intensive but solvable in polynomial time under practical constraints:

- The character recognition task (using CNNs) falls within the class **P (Polynomial Time)**. Once the model is trained, the prediction step—applying matrix operations and activations—runs in deterministic polynomial time with respect to the input image size.
- The spell-checking module, which uses edit distance and dictionary lookup, also belongs to class **P**, since dynamic programming techniques (e.g., Levenshtein distance) compute corrections in polynomial time with respect to the word length.
- However, the overall pipeline—especially when scaled for multi-script recognition, multi-word contextual corrections, and uncertainty resolution—can involve problems similar to those in the class of **NP-Complete**, such as optimal segmentation or parsing ambiguous handwriting in noisy data. These require heuristic or approximate solutions due to exponential branching.

Thus, while the individual components reside in $\mathbf{P}$, the combination and extension to large-scale multilingual document understanding could pose sub-problems that are intractable or **NP-Hard** in nature.

Modern Algebra Perspective

From an algebraic viewpoint, the character recognition task can be mapped onto a vector space over real numbers, where each image is treated as a tensor in $R^{m \times n}$. The model performs operations analogous to algebraic transformations—convolutions, activations, and weight multiplications—on this space.

The classification can be considered a function $f : R^{m \times n} \rightarrow C$, where C is the set of valid Devanagari character classes. These functions are trained to satisfy equivalence relations (i.e., two images representing the same character should map to the same class). The spell-checker uses group-like operations on word transformations, particularly in the use of edit-distance functions that operate under a cost metric, loosely similar to algebraic morphisms.

Hence, using modern algebra and formal language theory, the system's recognition and correction mechanisms can be modeled as compositions of functions and transformations on structured symbol sets, making them mathematically tractable and practically implementable.

10.1 Appendix B

10.1.1 Survey Paper Details

Paper Status: Published

Name of Journal: Bulletin for Technology and History (Volume 24 Issue 11 2024) ISSN No : 0391-6715

Title: A Survey on Machine Learning Models for Detecting Handwriting Errors in Devanagari Script

Page Numbers: 199-202

DOI: 10.37326/bthnlv24.11/1855

10.1.2 Implementation Paper Details

Paper Status: Applied

Name of Journal: Bulletin for Technology and History

Title: Handwritten Devanagari Character Recognition Using Deep Learning and Transfer Learning Approaches

10.2 Appendix C

10.2.1 Plagiarism Report

The plagiarism report for the project report has been attached herewith.

Similarity: 9%

CHAPTER 11

REFERENCES

- [1] Kumar, S. (2010). An analysis of irregularities in Devanagari script writing: A machine recognition perspective. *International Journal on Computer Science and Engineering*.
- [2] Jangid, M., & Srivastava, S. (2018). Handwritten Devanagari character recognition using layer-wise training of deep convolutional neural networks and adaptive gradient methods. *MDPI*.
- [3] Ali, A. A., & Mallaiah, S. (2022). Intelligent handwritten recognition using hybrid CNN architectures based-SVM classifier with dropout. *International Journal of Cognitive Computing in Engineering*.
- [4] Moudgil, A., Singh, S., Gautam, V., Rani, S., & Shah, S. H. (2022). Handwritten Devanagari manuscript characters recognition using CapsNet. *International Journal of Cognitive Computing in Engineering*.
- [5] Ajao, J. F., Olorunmaiye, E., & Mabayoje, M. A. (2022). Offline Yoruba word recognition system based on capsule neural network. *Ilorin Journal of Computer Science and Information Technology*.
- [6] Gupta, M. K., Lakshmi, C. V., & Patvardhan, C. (2016). Identification of character pattern in Devanagari words for enhancement of recognition accuracy. *Advances in Computer Science and Information Technology*.
- [7] Magdum, D., Maloji, S., & Suman. (2019). System for identifying and correcting invalid words in the Devanagari script for text to speech engine. *International Journal of Innovative Technology and Exploring Engineering (IJITEE)*.
- [8] Gupta, A., & Yadav, R. (2016). Recognition of Handwritten Devanagari Characters Using Support Vector Machine. *International Journal of Computer Science and Mobile Computing*, 5(4), 101–107.
- [9] Sarkar, S., & Roy, S. (2018). Deep Convolutional Neural Networks for Devanagari Handwritten Word Recognition. *Journal of Engineering and Technology*, 6(1), 28–35.
- [10] Singh, K., & Mehta, D. (2015). Character Segmentation and Recognition for Devanagari Script Using Morphological Operations. *International Journal of Computer Applications*, 117(3), 1–7.
- [11] Patel, S., & Patel, H. (2020). Machine Learning Approaches for Devanagari Handwritten Character Recognition. *Journal of Information Science and Engineering*, 36(3), 111–122.

- [12] Kumar, M., & Joshi, P. (2017). A Comparative Study of OCR Techniques for Devanagari Script Recognition. *International Journal of Advanced Computer Science and Applications*, 8(6), 21–28.
- [13] Bhattacharjee, S., & Dey, S. (2019). A Hybrid Approach to Devanagari Text Detection in Natural Scenes. *Computer Vision and Image Understanding*, 179, 24–34.

Paperpal Plagiarism Check Standard ⓘ

Overview

Similarity range

Similarity detected in your file!

3% - 9%

Sources for similarity

1

technodocbox.com
INTERNET

1%

2

www.adaface.com
INTERNET

1%

3

dokumen.pub
INTERNET

1%

A Survey on Machine Learning Models for Detecting Handwriting Errors in Devanagari Script

Darpan Kotagire, *Student*

Computer Engineering Department
Pune Institute of Computer Technology
Pune, India

Parth Dawande, *Student*

Computer Engineering Department
Pune Institute of Computer Technology
Pune, India

Sneha Basu, *Student*

Computer Engineering Department
Pune Institute of Computer Technology
Pune, India

Viraag Borundiya, *Student*

Computer Engineering Department
Pune Institute of Computer Technology
Pune, India

Samadhan Jadhav, *Asst. Prof.*

Computer Engineering Department
Pune Institute of Computer Technology
Pune, India

Achala Deshpande, *Asst. Prof.*

Computer Engineering Department
Pune Institute of Computer Technology
Pune, India

Abstract—This review paper surveys the current approaches in machine learning for detecting handwriting errors in the Devanagari script, a script used in several South Asian languages such as Hindi, Marathi, and Sanskrit. Due to the script's complex strokes and shapes, variations in handwriting often lead to misinterpretations and errors in recognition. This review examines various models, including Convolutional Neural Networks (CNNs), Capsule Networks (CapsNets), and hybrid architectures, highlighting their workflows, advantages, limitations, and future research directions. These models show promise in improving handwriting recognition systems, but challenges such as noisy data and varying handwriting styles remain significant.

Index Terms—Handwriting Recognition, Devanagari Script, Error Detection, Machine Learning, Convolutional Neural Networks, Capsule Networks, OCR

I. INTRODUCTION

The Devanagari script plays a crucial role in several South Asian languages, including Hindi, Marathi, and Sanskrit. Its complex nature, characterized by intricate strokes and unique structural components, often leads to errors in handwriting, which can severely impact communication and recognition in automated systems. Developing systems capable of accurately detecting and correcting these errors is vital for improving Optical Character Recognition (OCR) systems, educational tools, and document processing applications. As a result, numerous machine learning models have been developed to tackle this challenge, providing innovative approaches to improve handwriting recognition.

This review paper explores the different machine learning models that have been applied to error detection in handwritten Devanagari characters. The focus is on techniques such as Convolutional Neural Networks (CNNs), Capsule Networks (CapsNets), and hybrid architectures combining CNNs with Support Vector Machines (SVMs). These models have shown significant promise in improving the accuracy of handwriting recognition, particularly in dealing with the complexities of Devanagari script. However, the challenges of noisy data,

diverse handwriting styles, and limited datasets still pose significant hurdles to achieving optimal performance.

The structure of this review paper is as follows. First, it provides a theoretical overview of the machine learning techniques applied to handwriting recognition in Devanagari script, detailing the rationale behind focusing on error detection. Second, it elaborates on the methodologies used in the research studies covered, including the specific models such as CNNs, CapsNets, and hybrid architectures that were employed to analyze handwriting patterns. In the subsequent section, the findings from various studies are presented, highlighting the performance and limitations of these models. Afterwards, these findings are discussed in relation to the existing literature on handwriting recognition. Finally, the paper concludes with suggestions for future research directions to improve accuracy and model robustness.

By examining the current state of machine learning models in Devanagari handwriting recognition, this review aims to provide a comprehensive understanding of their strengths, limitations, and the ongoing efforts to address the challenges in the field. The insights gained from this review can pave the way for further advancements in both research and practical applications in handwriting recognition systems.

II. LITERATURE REVIEW

A. Devanagari Script and Error Detection

Satish Kumar [1] provides an early and insightful analysis of the irregularities encountered in Devanagari script writing and their effects on machine recognition systems. His approach involves a comprehensive workflow that includes scanning, preprocessing, feature extraction, and classification of the handwritten text. One of the notable contributions of Kumar's work is the segmentation of the Devanagari script into three vertical regions—upper, middle, and lower—which effectively helps in structuring the recognition process and managing the script's inherent irregularities.

While the vertical segmentation strategy offers a clear advantage in handling script variations, the approach still faces some significant challenges. Incomplete characters and inconsistent symbol sizes present a limitation, as these factors hinder the performance of intelligent character recognition (ICR) systems, making it difficult for models to accurately interpret the characters. To overcome these issues, Kumar suggests that more robust algorithms need to be designed. Additionally, there is a need for educating writers on improving the quality of their handwriting, which could further enhance the performance of handwriting recognition systems in future applications.

B. CNN-Based Devanagari Character Recognition

Mahesh Jangid and Sumit Srivastava [2] proposed a Convolutional Neural Network (CNN) model that achieved an impressive recognition accuracy of 97.65% on the V2DMDCHAR database. The model's workflow involves several key steps, including preprocessing of the input data, feature extraction through convolutional layers, and classification via fully connected layers. This structured approach allows the model to effectively recognize Devanagari characters, particularly in cases where the input is clear and well-defined.

However, while the model demonstrates high accuracy under ideal conditions, its performance significantly declines when faced with noisy or distorted images. The model's inability to handle variations in handwriting poses a notable limitation, impacting its robustness in real-world scenarios. To address these challenges, Jangid and Srivastava suggest that incorporating noise reduction techniques and augmenting the training dataset could help improve the model's ability to generalize and perform more consistently across diverse handwriting styles and noisy environments. This direction provides a potential pathway for enhancing the robustness of CNN-based handwriting recognition models.

C. Hybrid CNN-SVM for Handwriting Recognition

Amani Ali Ahmed Ali and Suresha Mallaiah [3] explored the effectiveness of a hybrid CNN-SVM model to improve handwritten text recognition, specifically for Arabic script. In this approach, the Convolutional Neural Network (CNN) is responsible for extracting features from the input data, while the Support Vector Machine (SVM) handles the classification task. To mitigate the issue of overfitting, techniques such as Zero Component Analysis (ZCA) and dropout were incorporated into the model. This combination leverages the feature extraction capabilities of CNNs along with the classification precision of SVMs, offering enhanced accuracy.

While the hybrid model demonstrates improved performance, it comes with certain limitations. The architecture requires a substantial amount of data to train effectively, making it data-intensive. Additionally, the model is computationally expensive, which could be a challenge for practical deployment, particularly in environments with limited resources. Despite these challenges, the authors suggest that this hybrid architecture could be adapted for Devanagari script recognition.

By focusing on data efficiency and optimizing computational demands, the model has the potential to significantly enhance handwriting recognition systems for Devanagari script as well.

D. Capsule Networks (CapsNet) for Handwritten Devanagari Recognition

Aditi Moudgil et al. [4] introduced a Capsule Network (CapsNet) model for recognizing handwritten Devanagari characters from manuscript images. One of the key strengths of CapsNet lies in its ability to preserve spatial hierarchies within the data, which is particularly beneficial for the intricate and complex shapes found in Devanagari script. This feature allows CapsNet to outperform traditional CNNs when it comes to handling distortions and variations in handwriting styles, making it an effective tool for recognizing Devanagari characters.

However, the model is not without its challenges. CapsNet requires a substantial amount of labeled training data to function effectively, which can be difficult to obtain, especially for diverse handwriting styles. Additionally, the model may struggle to generalize well to unseen handwriting variations, limiting its performance in practical applications. To address these limitations, future research could focus on exploring advanced data augmentation techniques and incorporating cursive forms of Devanagari script, which could significantly enhance the model's robustness and expand its application scope.

E. Offline Handwriting Recognition with CapsNet

Jumoke Falilat Ajao et al. [5] demonstrated the use of Capsule Networks (CapsNet) for offline Yoruba word recognition, achieving an accuracy of 85.49% in recognizing words that contain diacritic marks. This capability is particularly relevant to the challenges presented by Devanagari characters, which also include complex shapes and additional components like diacritics. The success of CapsNet in preserving spatial relationships within the characters suggests that this method could be effectively adapted to the recognition of Devanagari script as well.

Despite its advantages, the system faces limitations, primarily due to the availability of limited training data. This shortage of data hampers the model's ability to generalize effectively to new or unseen handwriting styles, thereby restricting its robustness in real-world applications. To overcome these challenges, future efforts could focus on adapting this approach for Devanagari script recognition, specifically addressing complex handwriting features through advanced data collection and augmentation techniques. This would help enhance the model's capacity to manage the intricacies of Devanagari script while maintaining high recognition accuracy.

F. Character Pattern Identification in Devanagari Words

Manoj Kumar Gupta et al. [6] focused on improving recognition accuracy in Devanagari handwriting by identifying specific patterns within Devanagari words, particularly single-letter words. Their study revealed that although there

are 345 commonly used symbols in the script, only 17 of these symbols frequently occur in the middle zone of single-letter words. By concentrating on these commonly occurring symbols, the recognition process becomes more streamlined, as the potential character set is narrowed down significantly, making the recognition of these characters more efficient and accurate.

While this method offers clear advantages in simplifying the recognition process, its focus on single-letter words limits its broader applicability. More complex words and phrases, which involve larger character sets and more intricate patterns, are not adequately addressed by this approach. For future work, expanding this method to cover two-letter words and more complex character patterns could enhance the overall efficiency and accuracy of recognition systems for Devanagari script, making them more applicable to a wider range of text complexities.

G. Text-to-Speech (TTS) Systems and Error Correction

Damodar Magdum et al. [7] proposed a system designed to correct invalid words in the Devanagari script, specifically for text-to-speech (TTS) engines. The system uses a state machine to detect and correct invalid characters based on predefined grammar rules, ensuring that the final output consists of valid Devanagari words. This correction process enhances the accuracy and clarity of speech synthesis by providing phonetic precision in TTS outputs.

While this system is primarily focused on speech applications, its underlying error correction mechanisms hold potential for broader use. The rule-based approach could be adapted to improve the robustness of handwriting recognition systems, addressing inaccuracies in character recognition. However, the current system's emphasis on speech limits its direct applicability to general handwriting recognition tasks. Future research could explore extending this approach to optical character recognition (OCR) and handwriting recognition systems. By incorporating these error-correction techniques, such systems could effectively address input errors beyond the domain of TTS, improving overall recognition accuracy.

III. DISCUSSION

The literature review on machine learning models and error correction approaches for detecting handwriting errors in Devanagari script reveals several important observations:

- **Model Performance Variability:** Convolutional Neural Networks (CNNs) demonstrate high recognition accuracy for clear inputs, but their performance significantly degrades with noisy or distorted inputs. Capsule Networks (CapsNets) offer better handling of spatial hierarchies and handwriting variations but require larger labeled datasets to generalize effectively.
- **Hybrid Approaches:** Combining CNNs with Support Vector Machines (SVMs) improves classification accuracy by leveraging the strengths of both architectures. However, such models are computationally expensive

and require large datasets for training, making them less practical for resource-constrained environments.

- **Rule-Based Systems:** Magdum et al. introduced a state machine for correcting invalid words in the Devanagari script, specifically for Text-to-Speech (TTS) systems. This method enhances the accuracy of TTS outputs by ensuring valid Devanagari syllables are generated. Although the system is tailored for speech synthesis, its error-correction mechanism can be adapted for general Optical Character Recognition (OCR) systems.
- **Character Pattern Recognition:** Gupta et al. focused on identifying frequent character patterns in Devanagari script, particularly for single-letter words. By narrowing the character set in the middle zone of these words, they simplified the recognition process and improved accuracy. This method, while effective for simpler cases, requires expansion to handle multi-letter words and more complex text.
- **Challenges in Handling Complex Features:** Both rule-based and machine learning models struggle with complex features in Devanagari script, such as diacritics and cursive forms. Incorporating advanced data augmentation techniques and noise reduction could improve the robustness of these models.
- **Data Requirements:** A recurring challenge in deep learning-based models is the need for substantial labeled data. Insufficient data hampers the generalization of CNNs, CapsNets, and hybrid models, particularly when dealing with varied handwriting styles.
- **Future Research Directions:** Future research should explore more robust models capable of handling noisy inputs and diverse handwriting styles. Rule-based approaches like Magdum et al.'s state machine can be integrated with machine learning models to develop hybrid systems. Extending recognition beyond individual characters to full words and sentences will further enhance the practical utility of these systems.

IV. CONCLUSION

In conclusion, this review has highlighted the advancements and challenges in using machine learning models for detecting handwriting errors in Devanagari script. While models like CNNs and CapsNets have shown great promise in feature extraction and classification, they face limitations in handling noisy data and diverse handwriting styles. Hybrid models that combine CNNs with SVMs have provided enhanced accuracy but at the cost of higher computational requirements. Additionally, methods focused on character pattern identification have simplified the recognition process for frequently occurring characters. However, future research must address challenges related to data scarcity and model robustness, especially in real-world applications. Expanding these models to recognize full words and sentences, along with improved data augmentation techniques, will be critical for the continued development of more effective handwriting recognition systems.

REFERENCES

- [1] Satish Kumar, "An Analysis of Irregularities in Devanagari Script Writing – A Machine Recognition Perspective," *International Journal on Computer Science and Engineering*, 2010.
- [2] Mahesh Jangid, Sumit Srivastava, "Handwritten Devanagari Character Recognition Using Layer-Wise Training of Deep Convolutional Neural Networks and Adaptive Gradient Methods," *MDPI*, 2018.
- [3] Amani Ali Ahmed Ali and Suresha Mallaiah, "Intelligent Handwritten Recognition Using Hybrid CNN Architectures Based-SVM Classifier with Dropout," *International Journal of Cognitive Computing in Engineering*, 2022
- [4] Aditi Moudgil, Saravjeet Singh, Vinay Gautam, Shalli Rani, Syed Hassan Shah, "Handwritten Devanagari Manuscript Characters Recognition Using CapsNet," *International Journal of Cognitive Computing in Engineering*, 2022.
- [5] Jumoke Falilat Ajao, Emmanuel Olorunmaiye, Modinat Abolore Mabayoje, "Offline Yoruba Word Recognition System Based on Capsule Neural Network," *Ilorin Journal of Computer Science and Information Technology*, 2022.
- [6] Manoj Kumar Gupta et al. (2016) - "Identification of Character Pattern in Devanagari Words for Enhancement of Recognition Accuracy"
- [7] Damodar Magdum et al. (2019) - "System for Identifying and Correcting Invalid Words in the Devanagari Script for Text to Speech Engine"